

COMP5008 Final Assignment

School of Electrical Engineering, Computing and Mathematical Sciences,
Curtin University

Title: Critical Care Optimisation: Data Structures and Algorithms for Hospital Efficiency

Introduction

Modern hospitals coordinate patients, staff, equipment, and space in real time. Algorithmic solutions can improve emergency response, reduce delays, and increase reliability.

In this assignment, you will design and implement a modular hospital resource management system using core data structures and algorithms. The system should demonstrate:

- Efficient storage and retrieval of patient information.
- Fast and reliable pathfinding between hospital departments.
- Priority-based scheduling of emergency cases.
- Sorting and reporting of patient records at the end of the day.

The assignment consists of four modules (Graph, Hash Table, Heap, Sorting). Each module builds upon the others to simulate hospital operations end-to-end, where:

Graph (Module 1): Provides travel time data between hospital units.

Hash Table (Module 2): Stores patient details and urgency levels for fast retrieval.

Heap (Module 3): Combines urgency (from Module 2) and time factors (from Module 1 or input data) to decide who gets treated next.

Sorting (Module 4): Takes all scheduled treatment records and organises them into sorted lists for efficiency analysis and reporting.

Learning Objectives

By completing this assignment, students will be able to:

- 1) Implement and analyse adjacency-list graphs, BFS, DFS, and a shortest path algorithm.
- 2) Design and evaluate a hash table with collision handling, suitable for 20+ records.
- 3) Implement a heap to schedule tasks by a computed priority metric and trace heap operations.
- 4) Implement merge sort and quick sort; benchmark and analyse performance under varying inputs.
- 5) Integrate multiple data structures into a coherent, testable system.

Module 1: Graph-Based Hospital Navigation

Problem Description

The hospital comprises departments such as Emergency, ICU, Pharmacy, Radiology, Laboratories, Operating Theatres, Wards, and Outpatient Units. Efficient navigation is essential for patient transfers and equipment movement.

Model the floor plan as a weighted, undirected graph:

- Nodes represent departments.
- Edges represent corridors/pathways between departments.
- Weights are walking times (minutes).

Tasks

1) Graph Construction

- Implement a Graph class using an adjacency list.
- Support dynamic insertion of departments (nodes) and corridors (weighted edges).
- Ensure undirected symmetry ($u \leftrightarrow v$ with same weight).

2) Algorithms to Implement (no built-in graph libraries):

- Breadth-First Search (BFS): Input = source department; Output = reachable departments grouped by level (hops).
- Depth-First Search (DFS): Input = source; Output = whether cycles exist and, if found, nodes involved.
- Shortest Path Algorithm: Implement A* Algorithm from a source; report path and total cost. Cite the algorithm source and implement from first principles without built-in shortest-path functions.

3) Test Case Setup

- At least 8 departments (nodes).
- 10–12 weighted corridors (edges).
- Include at least one cycle and one isolated department.

4) Expected Output

- Textual/visual graph structure.
- BFS results with levels (Level 0, Level 1, ...).
- DFS cycle detection (and cycle members if present).
- Shortest path and total walking time from the source department to a destination department.

Module 2: Hash-Based Patient Lookup

Problem Description

Clinicians require rapid access to patient records in emergencies. Implement a hash table to store and retrieve patient data efficiently, supporting at least 20 active records without significant performance degradation.

Tasks

1) Hash Table Design and Implementation (no built-in dict/Map for core logic):

- Use either chaining (linked lists) or open addressing (linear probing). Justify the choice in the report.
- Implement a simple modulo-based hash function; explain parameter choices (table size, prime selection).
- Define a Patient record with fields: PatientID, Name, Age, Department, UrgencyLevel (1–5), TreatmentStatus.

2) Core Operations:

- insert(record): $O(1)$ expected; handle duplicates (update or reject with message).
- search(patientID): $O(1)$ expected; return the full patient record or a not-found message.
- delete(patientID): $O(1)$ expected; confirm deletion and handle missing keys gracefully.

3) Collision Handling and Load Management:

- Demonstrate at least one collision scenario and show how it is resolved (probe sequence or chain state).
- Discuss and (optionally) implement resizing/rehashing when load factor exceeds a threshold (e.g., 0.7).
- Explain worst-case behaviors.

4) Validation and Error Handling:

- Validate inputs (e.g., urgency 1–5 only; non-empty names; IDs unique/integer).
- Ensure robustness against out-of-range indices and null references.

5) Testing Requirements:

- Insert at least 20 patient records (include diverse urgency levels and departments).

- Demonstrate successful searches (hits) and misses; demonstrate deletions and their effect on subsequent searches.
- Provide timing snapshots or operation counts to support $O(1)$ expected behaviour claims.

6) Expected Output:

- Log of inserts, searches, and deletes (concise, readable format).
- Explicit collision example(s) with intermediate states (e.g., probe indices or chain contents).
- Summary of complexity, load factor behaviour, and any resizing.

Deliverables for Module 2

- Source code for hash table (with comments and clear function headers).
- Test driver demonstrating inserts, searches, deletes, and collision handling.
- Short written justification of hash strategy, table size, and hash function.

Module 3: Heap-Based Emergency Scheduling

Problem Description

Emergency cases must be prioritised by urgency level and expected treatment time. Implement a scheduler using a heap that always returns the next highest-priority case.

Priority Metric

$$\text{Priority} = (6 - U) + 1000 / T$$

- **U = UrgencyLevel** (1 = highest priority, 5 = lowest).
- **T = Expected treatment time in minutes.**

Higher priority values should be treated first (Max Heap recommended).

Tasks

1) Heap Implementation (array-based binary heap; no use of built-in priority queues):

- Choose Max Heap (recommended) and justify; alternatively, use Min Heap with inverted keys.
- Implement: insert(request), peek(), extract_priority().
- Maintain heap invariants after each operation (percolate up/down).

2) Integration with Module 2 (Data Flow):

- Each request references a PatientID; retrieve UrgencyLevel and TreatmentStatus from the hash table.
- Use T (expected time) provided by scenario or by shortest-path estimates

from Module 1 where applicable.

- Compute the numeric priority for every request inserted.

3) Updates, and Edge Cases:

- Support updates (e.g., urgency level change) by re-inserting or providing a decrease/increase-key strategy (documented).
- Handle invalid PatientID, zero/negative treatment time, or inactive status gracefully.

4) Tracing and Observability:

- Print the heap array after each insert and each extract_priority.
- Log the computed priority and the inputs used (U and T) for each request.
- Demonstrate at least 10 inserts and 5 extractions in a reproducible test run.

Expected Output

- Sequence of heap states after operations (readable array/tree printout).
- Log of priority computations for each request.
- Evidence that higher-priority patients are consistently served first.

Deliverables for Module 3

- Heap source code with comments.
- Test driver producing the required logs and traces.
- Brief written note on update strategy.

Module 4: Sorting Patient Records

Problem Description

At the end of the day, patient records must be sorted for reporting and analysis. You will implement two comparison-based sorting algorithms and compare their performance.

Tasks

1) Implementations

- Merge Sort (top-down or bottom-up; justify choice).
- Quick Sort (choose pivot strategy; justify and discuss its effect).

Restrictions: Do not call language-provided sort utilities for core sorting logic.

- Sort by treatment duration (minutes).

2) Dataset Sizes and Input Conditions:

- Generate and sort datasets of size 100, 500, and 1000.
- For each size, test three conditions: random, nearly sorted ($\leq 10\%$ elements displaced), and reversed.
- Use deterministic random seeds for reproducibility.

3) *Analysis and Visualisation:*

- Present a table summarising times (and operation counts if collected).
- Provide a brief analysis: where does each algorithm excel or degrade and why (pivot choice, recursion depth, cache effects).

Expected Output

- Correctly sorted lists
- Timing table (and, if available, plots) comparing merge sort and quick sort across input sizes and conditions.
- Short written reflection on algorithm choice under different scenarios.

Deliverables for Module 4

- Source code for merge sort and quick sort.
- Benchmark script and results (tables and optional plots).
- Brief write-up on stability and pivot strategy.

Submission Guidelines

All students must submit the following components to complete the Final Assignment successfully. No group work and submissions are allowed for the final assignment. Ensure that your submission meets the formatting and content expectations outlined below.

Submission Deadline

- Due Date: **Wednesday, 22 October 2025, 11:59 PM AWST.** An additional 7 days for students with the Curtin Access Plan (CAP).

Submission Format

- All files must be submitted as a single ZIP file named:
COMP5008_Assignment_YourStudentID.zip

using the assignment submission link on Blackboard, which will include:

- Source code (Python or Java) with clear structure and comments.
- README with module-wise instructions to run/test each module.

- Sample input and output files with different case scenarios.
- Technical report in PDF format.

Source Code Requirements

- You may use Python/ Java. You can reuse the code from practical submissions and self-cite. For the shortest path algorithm, cite your algorithm reference.
- **Avoid using any built-in functions. A 100% penalty will be applied for using built-ins on any part where you apply a built-in function.**
- All code must be modular, with meaningful variable and function names.
- Include inline comments and a header block for each file.
- **No AI-generated code is accepted. 100% penalty will be applied if found.**
- Include error and exception checks in each module.

Technical Report Requirements

Use clear section headers, numbered pages, and consistent formatting throughout the report. Your PDF report must include:

- 1) Overview: System purpose and modular design.
- 2) Data Structures Used: Justification for graph, hash table, heap, sorting.
- 3) Module Integration: Flowchart/UML describing data flow and interactions.
- 4) Algorithm Complexity: Time and space analysis for each module.
- 5) Sample Output: Screenshots/log excerpts per module.
- 6) Reflection: Key learnings and design trade-offs.
- 7) Limitations & Assumptions: Known issues and simplifications.

Code Demonstration

- **Demo Date: 27th October – 7th November**
- **Demo Time:** Book a 20-minute demo slot (details will be provided on Blackboard).
- Be prepared to run each module live (in person/ online), explain code structure, and answer questions on correctness and complexity.
- **Demonstration is compulsory; non-attendance may result in no marks for the assignment. Be aware that final assignment consists of 40% of the overall grade and it is mandatory to participate in the assignment to meet the passing criteria.**

Academic Integrity

- All work must be your own. Discuss ideas, but do not share code.
- Cite any algorithmic references used (e.g., for A*, merge sort variants, pivot

strategies).

- Any evidence of plagiarism or unauthorised code generation may result in reporting to the university's higher authority and severe penalties.

Support

Feel free to reach out to your unit coordinator or Lecturer/ Tutors if you have any concerns. Post on Piazza regarding any inquiry related to the assignment.

Marking Criteria (Total: 40 Marks)

Module 1: Graph-Based Hospital Navigation (8 Marks)

- Graph construction and adjacency list design – *2 marks*
- Breadth-First Search (BFS): correct reachable departments and level grouping – *2 marks*
- Depth-First Search (DFS): correct cycle detection and reporting – *2 marks*
- Shortest Path Algorithm: from-scratch implementation A* and accurate output – *2 marks*

Module 2: Hash-Based Patient Lookup (6 Marks)

- Insert, search, and delete operations implemented correctly – *3 marks*
- Collision handling clearly demonstrated (probe sequence or chaining) – *1 mark*
- Effective hash function design and scalability for ≥ 20 patients – *2 marks*

Module 3: Heap-Based Emergency Scheduling (6 Marks)

- Insert and extract operations implemented correctly with heap property maintained – *2 marks*
- Correct priority score calculation using urgency level and treatment time – *2 marks*
- Heap state tracing/logging presented clearly (after inserts and extractions) – *2 marks*

Module 4: Sorting Patient Records (6 Marks)

- Correctness of Merge Sort and Quick Sort implementations; outputs validated – *3 marks*
- Performance comparison across dataset sizes and input conditions, with analysis – *3 marks*

Technical Report (8 Marks)

- Overview of system purpose and justification of chosen data structures – 2 *marks*
- Flowchart/UML showing module interactions and integration – 2 *marks*
- Time and space complexity analysis; reflection on performance and trade-offs – 2 *marks*
- Sample outputs/screenshots with assumptions and limitations documented – 2 *marks*

Code Quality & Compliance (6 Marks)

- Modular, well-structured, readable, and documented code – 2 *marks*
- Output traces showing sample inputs, including edge cases tested – 2 *marks*
- Proper error handling and adherence to restrictions (no built-ins, no AI code, exceptions handled) – 2 *marks*

Note:

- Demonstration is mandatory (in person or online). Non-attendance results in zero mark for the entire assignment.
- Plagiarism or AI-generated content: Any evidence will result in zero marks for affected sections or the entire assignment. Any use of built-ins on any part of the code will result in zero for that part.